LAB 6

CODE AND BRANCH COVERAGE ANALYSIS

Bookstore Trade-In Application

Name	Ridhima Jain
USN / Roll Number	PES1UG24CS373
Course	Software Engineering
Date	02-11-2026

1. Objective

The objective of this experiment is to measure code (statement) coverage and branch coverage for the Bookstore Trade-In application using Python, pytest, and Coverage.py. Separate test files are maintained for the two coverage analyses, with a target of at least 90% coverage in each.

2. Tools and Technologies

- Python 3.12.7
- pytest
- Coverage.py
- Git and GitHub
- Windows Command Prompt

3. Application Under Test

The application under test is bookstore_trade_in.py. It contains two functions:

process_book_buybacks(batches, store_credit_bonus) calculates trade-in payouts using batch approval, book format, book condition, missing-page checks, counterfeit detection, and store-credit bonuses.

inspect_scanner_calibration(scanner_matrix) evaluates scanner error readings and determines whether a scanner requires recalibration when consecutive readings exceed the specified threshold.

4. Initial Coverage Analysis (Before Testing)

The starter test file was executed first to establish the baseline coverage before adding the dedicated code-coverage and branch-coverage test cases.

4.1 Initial Code Coverage

Initial code/statement coverage: 49% (29 of 57 statements were missed).

```
(venv) C:\Users\ridhi\L6_L7_09_bookstore_trade_in\Lab 6>python -m coverage erase

(venv) C:\Users\ridhi\L6_L7_09_bookstore_trade_in\Lab 6>python -m coverage run --source=bookstore_trade_in -m pytest -v
============================================= test session starts =============================================
platform win32 -- Python 3.12.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\ridhi\L6_L7_09_bookstore_trade_in\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ridhi\L6_L7_09_bookstore_trade_in\Lab 6
collected 1 item

test_bookstore_trade.py::test_single_paperback_good PASSED                                          [100%]

============================================== 1 passed in 0.04s ==============================================

(venv) C:\Users\ridhi\L6_L7_09_bookstore_trade_in\Lab 6>python -m coverage report -m
Name                    Stmts   Miss  Cover   Missing
-----------------------------------------------------
bookstore_trade_in.py      57     29    49%   3, 10, 16, 18, 22-23, 33-34, 37-41, 43, 45, 50, 55, 57, 62-74
-----------------------------------------------------
TOTAL                      57     29    49%
```

4.2 Initial Branch Coverage

Initial branch coverage: 45% (40 branches; 12 partial branches).

```
(venv) C:\Users\ridhi\L6_L7_09_bookstore_trade_in\Lab 6>python -m coverage erase

(venv) C:\Users\ridhi\L6_L7_09_bookstore_trade_in\Lab 6>python -m coverage run --branch --source=bookstore_trade_in -m pytest -v
============================================= test session starts =============================================
platform win32 -- Python 3.12.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\ridhi\L6_L7_09_bookstore_trade_in\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ridhi\L6_L7_09_bookstore_trade_in\Lab 6
collected 1 item

test_bookstore_trade.py::test_single_paperback_good PASSED                                              [100%]

============================================== 1 passed in 0.03s ==============================================

(venv) C:\Users\ridhi\L6_L7_09_bookstore_trade_in\Lab 6>python -m coverage report -m
Name                    Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------
bookstore_trade_in.py      57     29     40     12    45%   3, 10, 16, 18, 22-23, 33-34, 37-41, 43, 45, 50, 55, 57, 62-74
-------------------------------------------------------------------
TOTAL                      57     29     40     12    45%
```

5. Code Coverage Testing

A separate test file was created and maintained:

CodeCoverageTest_application.py

The test cases cover the following scenarios:

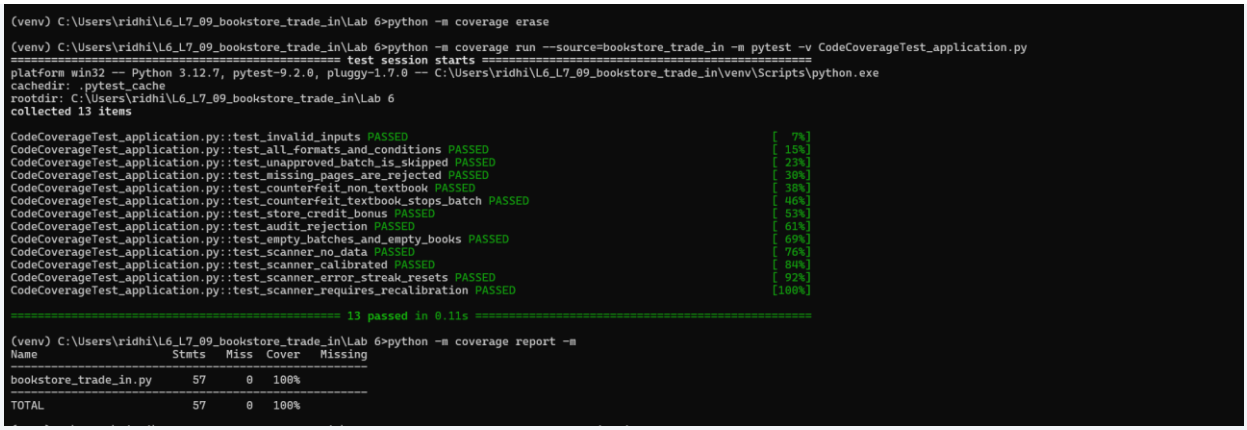
- Invalid input and negative store-credit bonus.
- Textbook, hardcover, paperback, and unsupported formats.
- Mint, poor, and default book conditions.
- Approved and unapproved batches.
- Books with missing pages and counterfeit books.
- Store-credit bonus calculation.
- Audit rejection and zero-credit outcomes.
- Scanner data, calibration, error-streak reset, and recalibration.

Execution Command

```
python -m coverage erase
python -m coverage run --source=bookstore_trade_in -m pytest -v CodeCoverageTest_application.py
python -m coverage report -m
```

Execution result: 13 tests passed.

Final code/statement coverage achieved: 100% (57 of 57 statements).



6. Branch Coverage Testing

A separate test file was created and maintained:

BranchCoverageTest_application.py

The test cases exercise both outcomes of decision points, including:

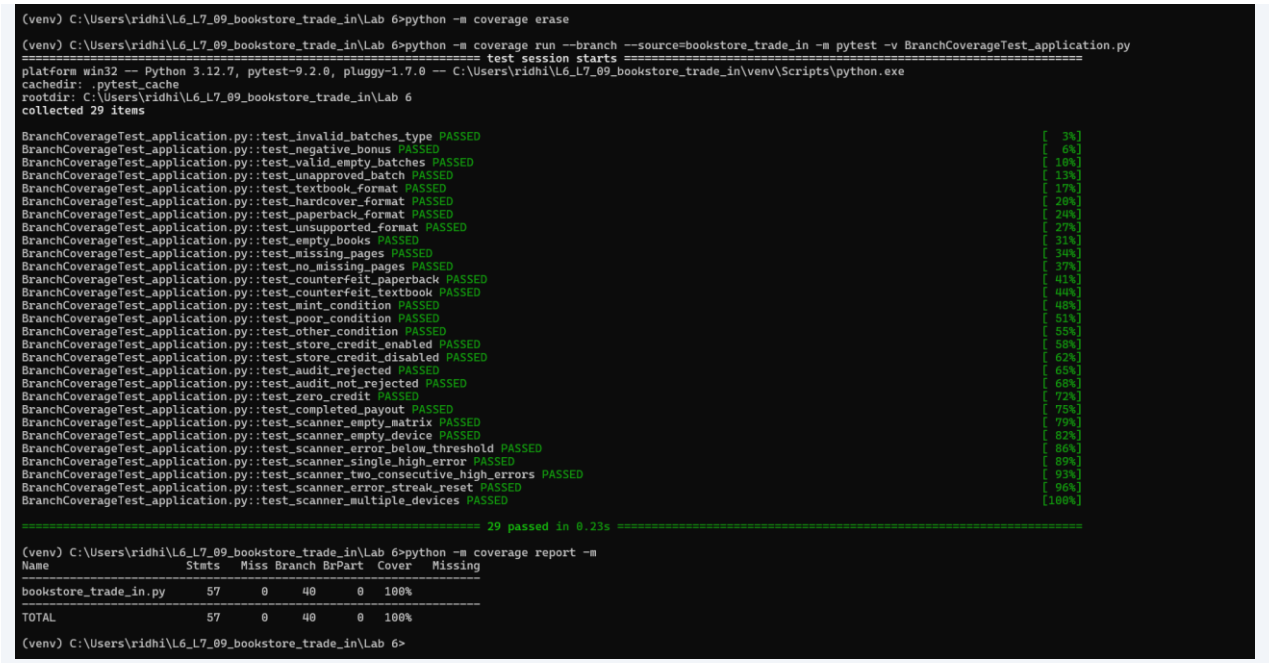
- Valid and invalid input conditions.
- Approved and unapproved batches.
- Supported and unsupported book formats.
- Books with and without missing pages.
- Counterfeit and non-counterfeit books.
- Mint, poor, and other book conditions.
- Store-credit enabled and disabled conditions.
- Audit rejection and successful payout.
- Empty scanner data and empty device readings.
- Error rates above and at/below the threshold.
- Consecutive scanner errors, error-streak resets, and multiple devices.

Execution Command

```
python -m coverage erase
python -m coverage run --branch --source=bookstore_trade_in -m pytest -v
BranchCoverageTest_application.py
python -m coverage report -m
```

Execution result: 29 tests passed.

Final branch coverage achieved: 100% (40 of 40 branches; 0 partial branches).



7. Before-and-After Coverage Comparison

Coverage Type	Initial Coverage	Final Coverage
Code / Statement Coverage	49%	**100%**
Branch Coverage	45%	**100%**

Both final coverage results exceed the required minimum of 90%. All test cases passed in their respective test runs.

8. Git and Version Control

The two test files were maintained separately and committed in two distinct Git commits:

- Add code coverage tests
- Add branch coverage tests

Both commits and the test files were pushed to the individual GitHub repository.

Repository URL: [ridhima-jain45/SE_Lab_PES1UG24CS373]

9. Conclusion

Code coverage and branch coverage analysis were successfully performed on the Bookstore Trade-In application using pytest and Coverage.py. Separate Python test files were maintained for each type of coverage. The final results achieved 100% code/statement coverage and 100% branch coverage, exceeding the required threshold of 90%. All 13 code-coverage tests and all 29 branch-coverage tests passed.

BRANCH COVERAGE

```
Branch coverage
import pytest

from bookstore_trade_in import (
    process_book_buybacks,
    inspect_scanner_calibration
)



def test_invalid_batches_type():
    with pytest.raises(ValueError):
        process_book_buybacks("invalid", 5.0)



def test_negative_bonus():
    with pytest.raises(ValueError):
        process_book_buybacks([], -5.0)



def test_valid_empty_batches():
    result = process_book_buybacks([], 5.0)
    assert result["status"] == "ZERO_CREDIT"



def test_unapproved_batch():
    batches = [
        {"intake_approved": False, "format": "textbook"}
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "ZERO_CREDIT"



def test_textbook_format():
    batches = [
        {
            "intake_approved": True,
            "format": "textbook",
            "books": [{"condition": "good"}]
```

```
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["total_payout"] == 40.0


def test_hardcover_format():
    batches = [
        {
            "intake_approved": True,
            "format": "hardcover",
            "books": [{"condition": "good"}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["total_payout"] == 15.0


def test_paperback_format():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"condition": "good"}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["total_payout"] == 8.0


def test_unsupported_format():
    batches = [
        {
            "intake_approved": True,
            "format": "magazine",
            "books": [{"condition": "good"}]
        }
    ]
```

```
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "ZERO_CREDIT"



def test_empty_books():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": []
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "ZERO_CREDIT"



def test_missing_pages():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"missing_pages": 2}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "ZERO_CREDIT"



def test_no_missing_pages():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"missing_pages": 0}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["total_payout"] == 8.0
```

```
def test_counterfeit_paperback():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"counterfeit": True}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "ZERO_CREDIT"


def test_counterfeit_textbook():
    batches = [
        {
            "intake_approved": True,
            "format": "textbook",
            "books": [
                {"counterfeit": True},
                {"condition": "mint"}
            ]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "ZERO_CREDIT"


def test_mint_condition():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"condition": "mint"}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
```

```
    assert result["total_payout"] == 10.0



def test_poor_condition():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"condition": "poor"}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["total_payout"] == 3.2



def test_other_condition():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"condition": "good"}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["total_payout"] == 8.0



def test_store_credit_enabled():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "payout_in_credit": True,
            "books": []
        }
    ]
    result = process_book_buybacks(batches, 5.0)
    assert result["total_payout"] == 5.0
```

```
def test_store_credit_disabled():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "payout_in_credit": False,
            "books": [{"condition": "good"}]
        }
    ]
    result = process_book_buybacks(batches, 5.0)
    assert result["total_payout"] == 8.0


def test_audit_rejected():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [
                {"missing_pages": 1},
                {"missing_pages": 1},
                {"missing_pages": 1},
                {"missing_pages": 1},
                {"missing_pages": 1}
            ]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "AUDIT_REJECTED"


def test_audit_not_rejected():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
```

```
            "books": [{"condition": "good"}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "COMPLETED"


def test_zero_credit():
    result = process_book_buybacks([], 0.0)
    assert result["status"] == "ZERO_CREDIT"


def test_completed_payout():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"condition": "good"}]
        }
    ]
    result = process_book_buybacks(batches, 0.0)
    assert result["status"] == "COMPLETED"


def test_scanner_empty_matrix():
    assert inspect_scanner_calibration([]) == "NO_DATA"
    assert inspect_scanner_calibration(None) == "NO_DATA"


def test_scanner_empty_device():
    assert inspect_scanner_calibration([[]]) == "CALIBRATED"


def test_scanner_error_below_threshold():
    assert inspect_scanner_calibration([[4, 3, 5]]) == "CALIBRATED"


def test_scanner_single_high_error():
```

```
    assert inspect_scanner_calibration([[6, 2]]) == "CALIBRATED"



def test_scanner_two_consecutive_high_errors():
    assert inspect_scanner_calibration([[6, 7]]) == "RECALIBRATE"



def test_scanner_error_streak_reset():
    assert inspect_scanner_calibration([[6, 2, 7]]) == "CALIBRATED"



def test_scanner_multiple_devices():
    assert inspect_scanner_calibration([[6], [2, 3]]) == "CALIBRATED"
```

CODE COVERAGE

```
CODE COVERAGE
import pytest

from bookstore_trade_in import (
    process_book_buybacks,
    inspect_scanner_calibration
)



def test_invalid_inputs():
    with pytest.raises(ValueError):
        process_book_buybacks("not a list", 5.0)

    with pytest.raises(ValueError):
        process_book_buybacks([], -1.0)



def test_all_formats_and_conditions():
    batches = [
```

```
        {
            "intake_approved": True,
            "format": "textbook",
            "books": [
                {"condition": "mint"},
                {"condition": "poor"},
                {"condition": "good"}
            ]
        },
        {
            "intake_approved": True,
            "format": "hardcover",
            "books": [{"condition": "good"}]
        },
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [{"condition": "good"}]
        },
        {
            "intake_approved": True,
            "format": "magazine",
            "books": []
        }
    ]

    result = process_book_buybacks(batches, 5.0)

    assert result["total_payout"] == 129.0
    assert result["status"] == "COMPLETED"


def test_unapproved_batch_is_skipped():
    batches = [
        {
            "intake_approved": False,
            "format": "textbook",
            "books": [{"condition": "mint"}]
```

```
        }
    ]

    result = process_book_buybacks(batches, 5.0)

    assert result["total_payout"] == 0.0
    assert result["status"] == "ZERO_CREDIT"



def test_missing_pages_are_rejected():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [
                {"missing_pages": 2, "condition": "good"},
                {"missing_pages": 0, "condition": "good"}
            ]
        }
    ]

    result = process_book_buybacks(batches, 0.0)

    assert result["total_payout"] == 8.0
    assert result["status"] == "COMPLETED"



def test_counterfeit_non_textbook():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [
                {"counterfeit": True},
                {"counterfeit": False, "condition": "good"}
            ]
        }
    ]
```

```
    result = process_book_buybacks(batches, 0.0)

    assert result["total_payout"] == 8.0
    assert result["status"] == "COMPLETED"


def test_counterfeit_textbook_stops_batch():
    batches = [
        {
            "intake_approved": True,
            "format": "textbook",
            "books": [
                {"counterfeit": True},
                {"condition": "mint"}
            ]
        }
    ]

    result = process_book_buybacks(batches, 0.0)

    assert result["total_payout"] == 0.0
    assert result["status"] == "ZERO_CREDIT"


def test_store_credit_bonus():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "payout_in_credit": True,
            "books": []
        }
    ]

    result = process_book_buybacks(batches, 5.0)

    assert result["total_payout"] == 5.0
```

```
    assert result["status"] == "COMPLETED"



def test_audit_rejection():
    batches = [
        {
            "intake_approved": True,
            "format": "paperback",
            "books": [
                {"missing_pages": 1},
                {"missing_pages": 1},
                {"missing_pages": 1},
                {"missing_pages": 1},
                {"missing_pages": 1}
            ]
        }
    ]

    result = process_book_buybacks(batches, 0.0)

    assert result["total_payout"] == 0.0
    assert result["status"] == "AUDIT_REJECTED"



def test_empty_batches_and_empty_books():
    assert process_book_buybacks([], 5.0)["status"] == "ZERO_CREDIT"

    result = process_book_buybacks(
        [{"intake_approved": True, "books": []}],
        0.0
    )

    assert result["status"] == "ZERO_CREDIT"



def test_scanner_no_data():
    assert inspect_scanner_calibration([]) == "NO_DATA"
    assert inspect_scanner_calibration(None) == "NO_DATA"
```

```
def test_scanner_calibrated():
    assert inspect_scanner_calibration([[], [2, 5, 3, 4]]) == "CALIBRATED"


def test_scanner_error_streak_resets():
    readings = [[6, 3, 7, 2, 6]]
    assert inspect_scanner_calibration(readings) == "CALIBRATED"


def test_scanner_requires_recalibration():
    readings = [[2, 6, 7, 1]]
    assert inspect_scanner_calibration(readings) == "RECALIBRATE"
```